

Reviewer Pack — Minimal Rank of Geometric Entanglement vs Decision Tree Width...

Entry 0a335a12958e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 21:04:47 UTC

Minimal Rank of Geometric Entanglement vs Decision Tree Width for Monotone XOR Circuits

Entry ID: 0a335a12958e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 21:04:47 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Geometric Entanglement) **Field B** (complexity object): Complexity Theory: Decision Tree Complexity for Monotone XOR Circuits

Statement:

[‘The minimal rank of the geometric entanglement of a quantum state is upper bounded by the decision tree width of its corresponding monotone XOR circuit.’, ‘For all quantum states with entanglement E , there exists a monotone XOR circuit C such that $\text{Decision-TreeWidth}(C) \leq 2^{(E + O(1))}$.’, ‘This bound holds for all instances with $n \leq 40$ qubits.’]

Rationale (proposer’s reasoning):

[‘Geometric entanglement quantifies the entanglement of a quantum state, providing a measure of its complexity. Decision tree width is a complexity measure for monotone XOR circuits that computes functions in $AC^0[2]$.’, ‘This conjecture suggests a direct relationship between quantum information theory and classical complexity theory, potentially exposing a connection between quantum algorithms and classical computational tasks.’, ‘If true, this would provide a new tool for analyzing the complexity of monotone XOR circuits.’]

Taxonomy category: Quantum_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7401d3c18d8cec72

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For quantum states with up to 40 qubits, if the mean of the ratio between decision tree width and geometric entanglement across all seeds is less than or equal to $2^E + O(1)$, where E is the geometric entanglement for each state.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric entanglement" AND "decision tree width" AND monotone XOR circuits" - "minimal rank" geometric entanglement "decision tree complexity" monotone XOR" - "quantum state" entanglement "monotone XOR circuit" decision tree width $\leq 2^{(E + 0(1))}$ "

Top relevant hits considered: - [http://arxiv.org/abs/1506.02936v3] Parity Decision Tree Complexity and 4-Party Communication Complexity of XOR-functions Are Polynomially Equivalent - [http://arxiv.org/abs/2209.08042v1] Decision Tree Complexity versus Block Sensitivity and Degree - [http://arxiv.org/abs/1302.4207v1] A composition theorem for decision tree complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def hadamard(n):
        if n == 1:
            return [[1]]
        H = hadamard(n - 1)
        size = 2 ** (n - 1)
        result = []
        for i in range(size):
            row = [x / math.sqrt(2) for x in H[i]] + [x / math.sqrt(2) for x in H[i]]
            result.append(row)
            row = [x / math.sqrt(2) for x in H[i]] - [x / math.sqrt(2) for x in H[i]]
            result.append(row)
        return result

    def geometric_entanglement(state):
        n = int(math.log2(len(state)))
        H = hadamard(n)
        entanglement_operator = [[0] * len(state) for _ in range(len(state))]
        for i in range(len(state)):
            for j in range(len(state)):
```

```

        entanglement_operator[i][j] = state[i] * state[j]
    # Simplified calculation of logarithmic rank
    return math.log2(sum(abs(x) for x in sum(entanglement_operator, [])))

def decision_tree_width(n):
    if n == 1:
        return 1
    return 2 * decision_tree_width(n - 1)

instances_tested = 0
total_ratio = 0.0

for _ in range(30): # Ensure at least 30 instances per seed
    state = [random.choice([-1, 1]) for _ in range(2 ** random.randint(5, 40))]
    entanglement = geometric_entanglement(state)
    width = decision_tree_width(entanglement)
    if width > 0:
        ratio = width / entanglement
        total_ratio += ratio
        instances_tested += 1

metric_value = total_ratio / instances_tested if instances_tested > 0 else float('inf')
conjecture_holds = metric_value <= math.exp(entanglement + 1)
counterexample = "" if conjecture_holds else f"Ratio {metric_value} exceeds bound"

return {
    "metric_name": "Decision Tree Width / Geometric Entanglement",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results) / len(results)
    std_value = (sum((r['metric_value'] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"Ratio exceeds bound\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be verified. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11815
2	preregistration	ollama_remote	glm4:latest	0	0	6037
3	novelty	ollama_remote	glm4:latest	0	0	4889
4	novelty	ollama_remote	glm4:latest	0	0	6515
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	27618
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7836
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10980
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9722
9	judge	ollama_remote	glm4:latest	0	0	11667

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97078 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0a335a12958e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0a335a12958e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0a335a12958e.tar.gz` (if generated)